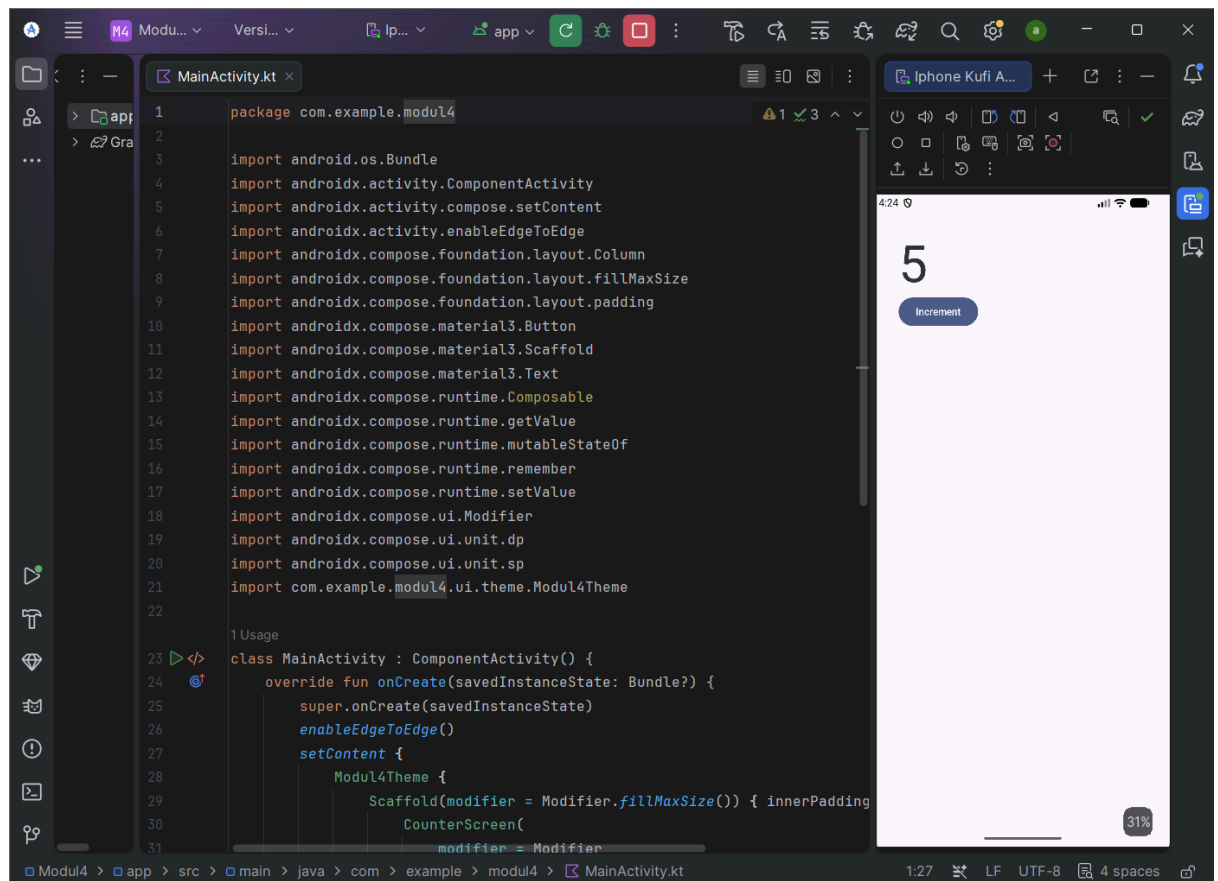


Nama: Alchalisa Fathinna
NIM: 245150407111066
MK: Pengembangan Aplikasi Mobile - F

State Hoisting

1. Jalankan program, pastikan aplikasi dapat dijalankan dengan baik pada emulator atau perangkat Android fisik. Kemudian klik tombol Increment; beberapa kali, ambil satu screenshot setelah tombol diklik beberapa kali.



a. Apa yang terjadi setiap kali tombol Increment diklik?

Jawaban: Angka yang ditampilkan di layar akan bertambah 1

b. Apa yang menyebabkan hal tersebut terjadi?

Jawaban: Event onClick pada Button memicu blok kode `number++`, yang mana mengubah nilai variabel state, sehingga memicu Jetpack Compose melakukan proses *recomposition* (menggambar ulang antarmuka)

c. Variabel state apa yang berperan pada Composable Text yang menampilkan perubahan angka pada Text tersebut?

Jawaban: Variabel `number`

d. Variabel state apa yang berperan untuk menampilkan label teks pada Composable Button yang bertuliskan Increment tersebut?

Jawaban: Variabel `label`

2. Pada percobaan tahap sebelumnya, ubah kode program pada Composable CounterScreen agar setiap kali tombol Increment diklik, nilai yang ditampilkan menjadi increment 2 (bertambah 2 setiap klik). Ubah pula label tombol Increment; menjadi Increment2.

a. Perubahan apa yang perlu dilakukan dan dilakukan di mana?

Jawaban: Perubahan dilakukan di dalam fungsi CounterScreen.

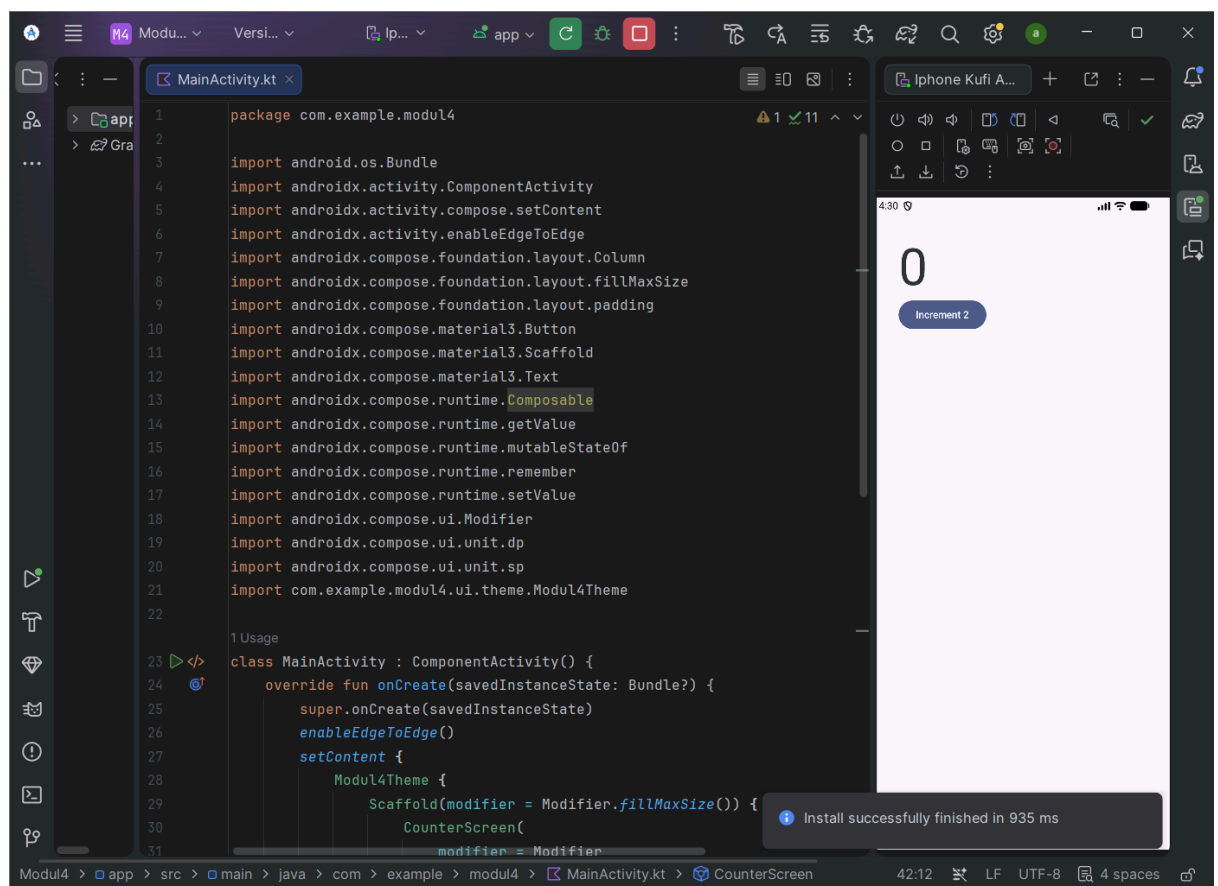
Ubah nilai awal label menjadi: `val label by remember { mutableStateOf("Increment 2") }`

Ubah logika di dalam Button onClick menjadi: `Button(onClick = { number += 2 })`

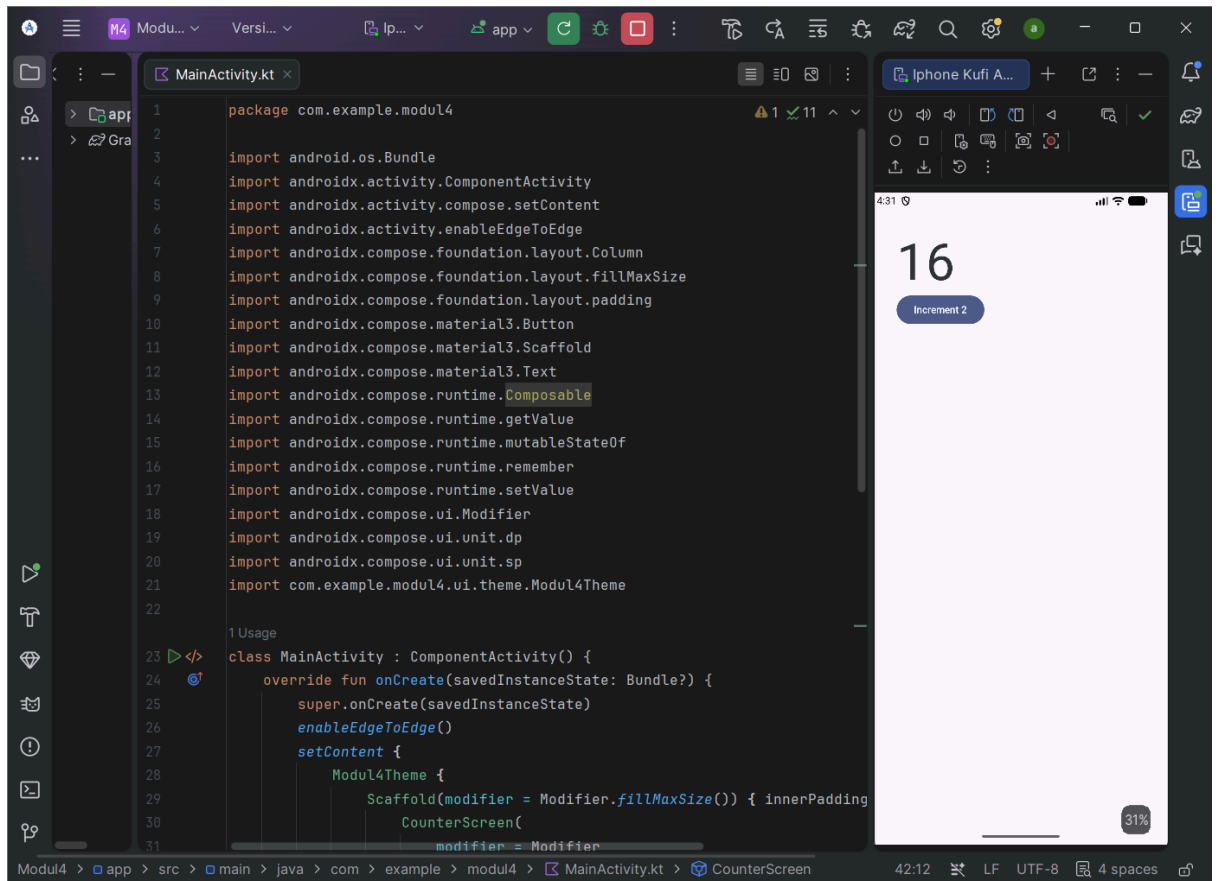
b. Jalankan program dan ambil masing-masing satu screenshot sebelum dan sesudah tombol Increment diklik untuk memperlihatkan bahwa perubahan tersebut berjalan dengan baik.

Jawaban:

Sebelum



Sesudah



- Langkah selanjutnya adalah melakukan state dan event hoisting pada state dan event yang sebelumnya terdapat di dalam Composable CounterScreen. Ubah signature kode program pada bagian fungsi Composable CounterScreen dari yang sebelumnya:

`fun CounterScreen(modifier: Modifier)`

menjadi:

`fun CounterScreen(modifier: Modifier,
number: Int, label: String, onClick: () -> Unit)`

Ubah body di dalam Composable CounterScreen menjadi:

```
// var number by remember { mutableStateOf(0) }
// val label by remember { mutableStateOf("Increment") }
Column(modifier) {
    Text(text = "$number", fontSize = 72.sp)
    Button(onClick = onClick) {
        Text(text = "$label")
    }
}
```

Ubah pemanggilan fungsi Composable CounterScreen di dalam fungsi onCreate() dari yang sebelumnya:

```
CounterScreen(modifier = Modifier.padding(32.dp))
```

menjadi:

```
var number by remember { mutableStateOf(0) }
```

```
CounterScreen(Modifier.padding(32.dp),
```

```
number = number,
```

```
label = "Increment"
```

```
onButtonClick = { number++ }  
)
```

Perhatikan bahwa deklarasi variabel state `number` dari yang sebelumnya dilakukan di dalam fungsi `Composable CounterScreen`, sekarang dideklarasikan di dalam fungsi `onCreate()` sebelum fungsi `Composable CounterScreen` dipanggil dari dalam fungsi `onCreate()`.

4. Jalankan program, dan kemudian klik beberapa kali tombol `Increment` sehingga angka yang ditampilkan berubah.

- a. Apakah program berjalan dengan baik serta menampilkan hal yang sama persis seperti yang dilakukan pada tahap nomor 4 sebelumnya?

Jawaban: Ya, program berjalan dengan baik dan fungsinya sama persis dengan tahap 4

- b. Di bagian kode program manakah saat ini proses `increment` dilakukan?

Jawaban: Proses *increment* dilakukan di dalam pemanggilan parameter fungsi `onButtonClick` di dalam fungsi `onCreate()`

- c. Di bagian kode program manakah saat ini state `Composable Text` dan `Composable Button "Increment"` dideklarasikan?

Jawaban: Di dalam fungsi induknya, yaitu di dalam `onCreate()`

5. Diasumsikan terdapat perubahan kebutuhan fungsional aplikasi yang memaksa terjadinya perubahan logika berjalannya program pada antar muka pengguna. Ubah kode program setelah tahap nomor 7 dilakukan sedemikian rupa sehingga:

- a. Label tombol `"Increment"` menjadi `"Double"`;

- b. Nilai awal variabel `number` dari 0 menjadi 1.

- c. Ubah fungsi/lambda `onButtonClick` pada pemanggilan `Composable CounterScreen` di dalam fungsi `onCreate()` sedemikian rupa sehingga yang dilakukan dari sebelumnya: `number++` menjadi menggandakan nilai variabel `number`.

- d. Tuliskan bagian kode program yang mendeklarasikan state dan pemanggilan fungsi `Composable CounterScreen` di dalam fungsi `onCreate()` setelah langkah nomor 8(a), 8(b), dan 8(c) dilakukan.

Jawaban:

```
var number by remember { mutableStateOf(1) }
```

```
CounterScreen(  
    modifier = Modifier.padding(32.dp),
```

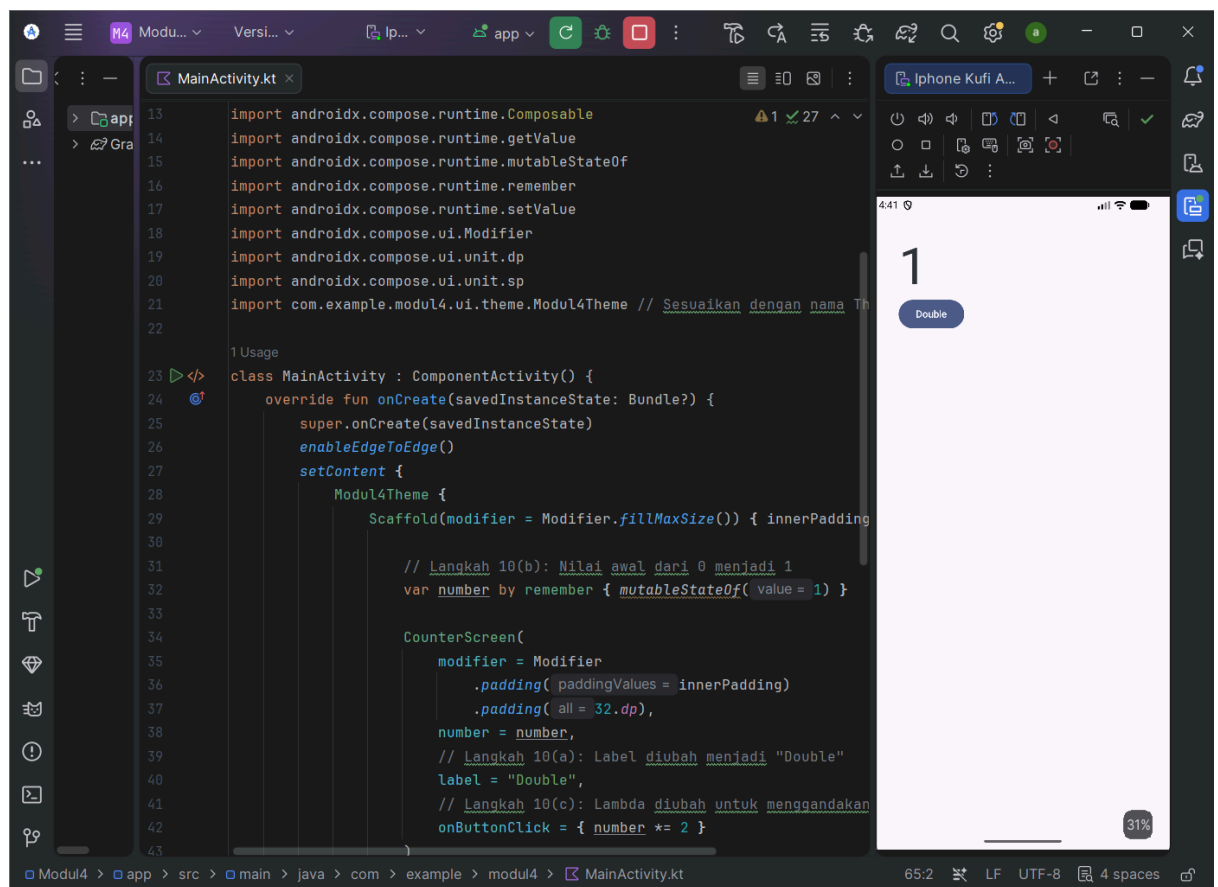
```
number = number,  
label = "Double", // 8(a) Label diubah  
onButtonClick = { number *= 2 }  
)
```

6. Jalankan program,

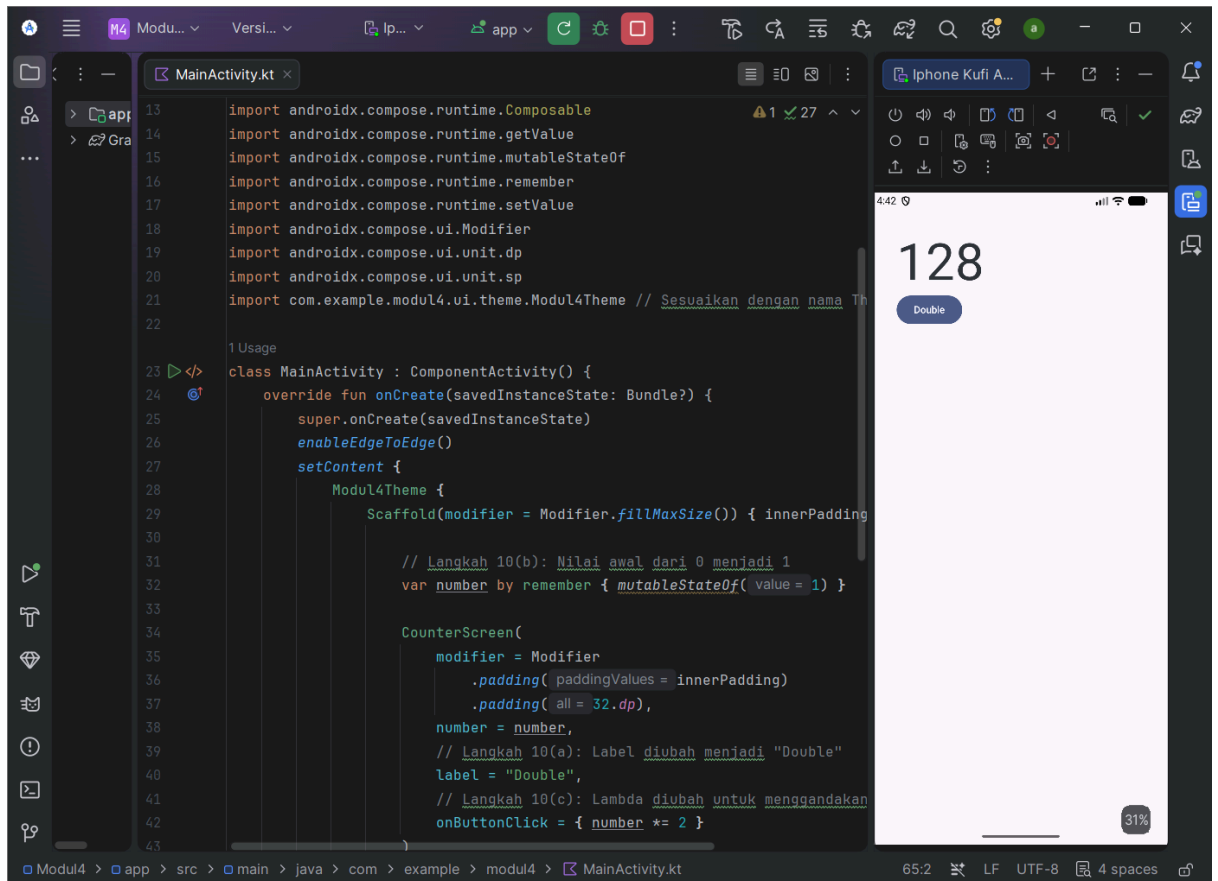
a. Perlihatkan dengan mengambil satu screenshot sebelum dan sesudah nilai variabel number digandakan.

Jawaban:

Sebelum



Sesudah



b. Di bagian manakah dari kode program pada percobaan tahap nomor 7 dan tahap nomor 8 dilakukan?

Jawaban: Dilakukan di dalam fungsi induk yang memanggilnya, yaitu di dalam fungsi `onCreate()`

c. Apakah ada perubahan yang dilakukan dari tahap nomor 7 dan tahap nomor 8 yang dilakukan di dalam fungsi Composable `CounterScreen` itu sendiri? Jika ada sebutkan perubahan yang dilakukan.

Jawaban: Tidak ada perubahan logika sama sekali di dalam fungsi `CounterScreen` setelah tahap 7 dan 8 dilakukan

7. Apa arti dari jawaban pada tahap nomor 9 terkait dengan manfaat dari melakukan state hoisting dalam perubahan kebutuhan fungsional aplikasi yang terkait dengan antar muka pengguna?

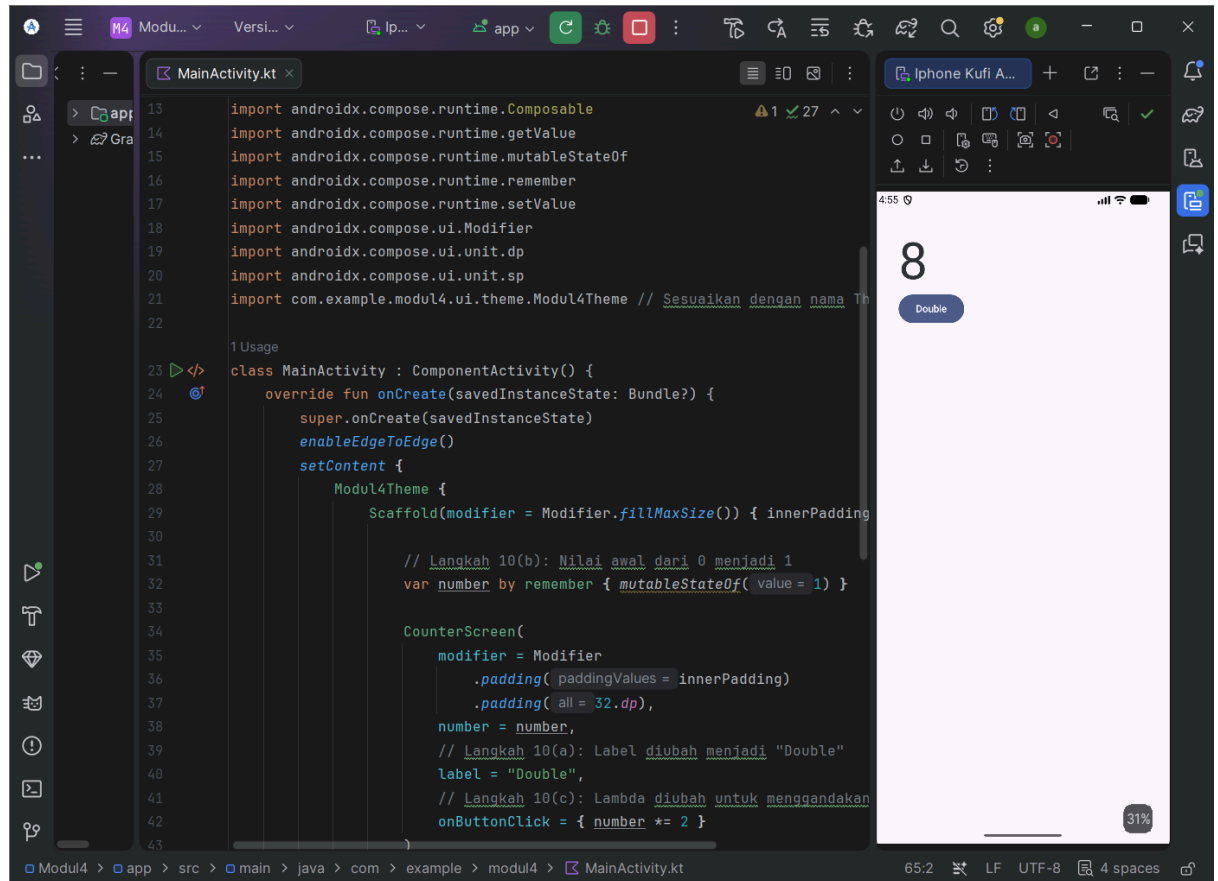
Jawaban: Membuat UI Composable menjadi *stateless* (tidak menyimpan data sendiri) sehingga mudah digunakan kembali (*reusable*) dan mengikuti prinsip *Single Source of Truth (SSoT)* di mana data dan logika bisnis dikontrol oleh komponen induk

Perubahan Konfigurasi

1. Jalankan program, dengan memastikan posisi perangkat pada posisi **portrait**, kemudian klik beberapa kali tombol "Double" sedemikian rupa sehingga angka yang

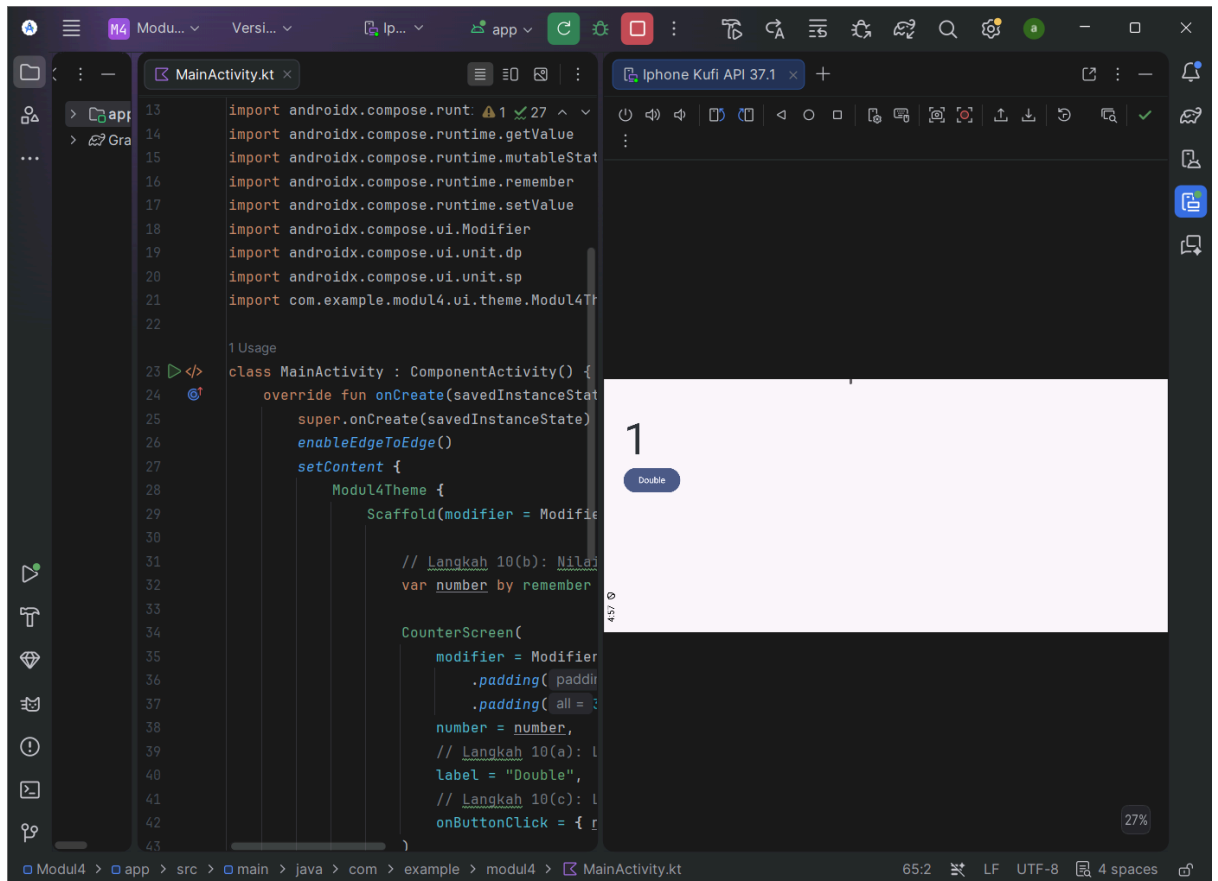
ditampilkan tidak lagi angka 1 yang merupakan nilai awal yang diset pada variabel number saat program pertama kali dijalankan.
Catat angka yang ditampilkan setelah tombol "Double" diklik beberapa kali, ambil screenshot tampilan antarmukanya.

Jawaban:



2. Lakukan perubahan konfigurasi dengan memutar posisi perangkat dari posisi *portrait* menjadi *landscape*, sehingga tampilan aplikasi akan disesuaikan oleh Android.

Jawaban:



- a. Catat angka yang ditampilkan kemudian ambil screenshot tampilan antar muka aplikasi di posisi landscape.

Jawaban: Angka yang ditampilkan kembali menjadi 1.

- b. Berapa angka yang ditampilkan? Mengapa terjadi demikian?

Jawaban: Perubahan orientasi layar memicu sistem Android untuk menghancurkan (*destroy*) dan membuat ulang (*recreate*) *Activity* dari awal. Karena variabel `number` masih menggunakan konstruk `remember` biasa, data tersebut ikut hilang saat *Activity* dihancurkan dan otomatis mereset nilainya ke angka awal, yaitu 1.

Ubah bagian kode program di dalam fungsi `onCreate()` seperti berikut ini:

```
var number by remember { mutableStateOf(1) }
```

menjadi:

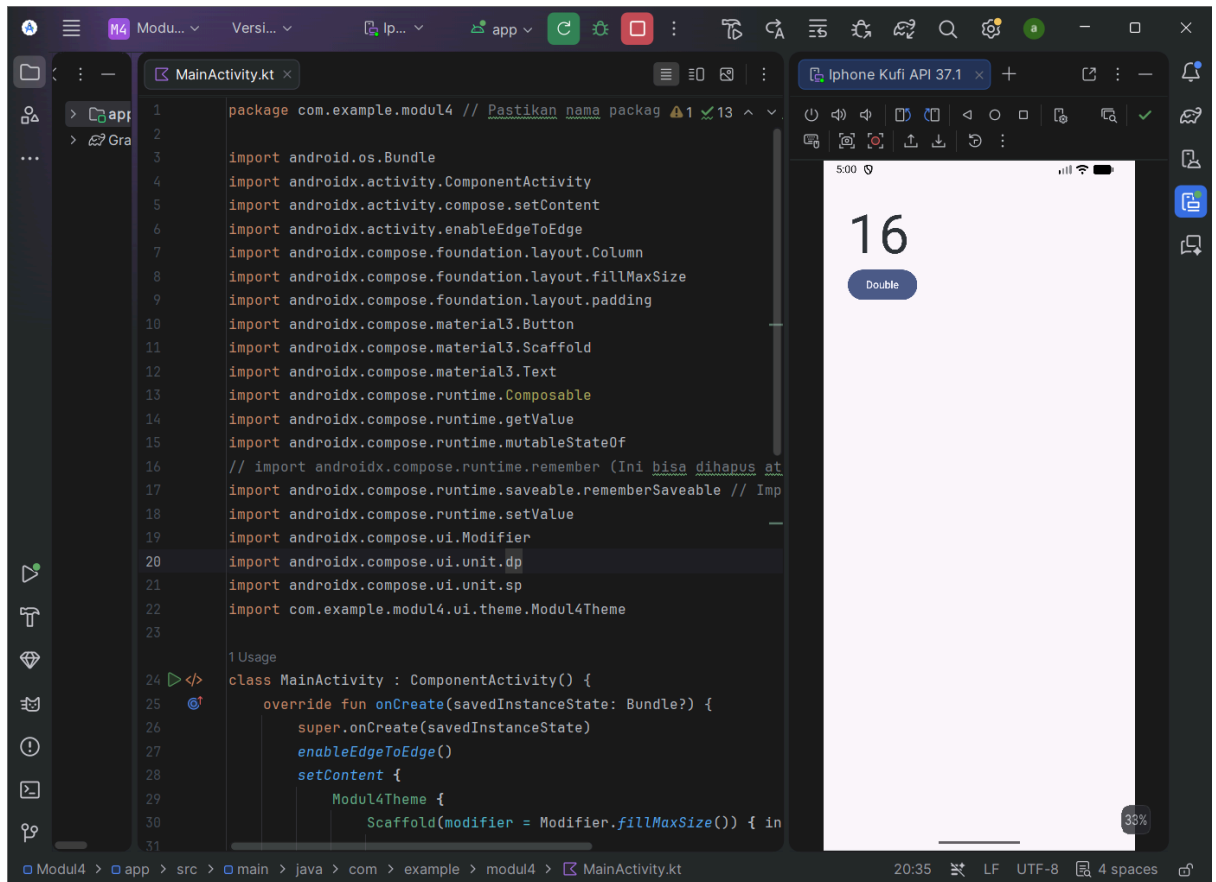
```
var number by rememberSaveable { mutableStateOf(1) }
```

Lakukan *import dependency* atas perubahan konstruk `remember` menjadi `rememberSaveable` yang dilakukan (dengan memposisikan kursor pada konstruk `rememberSaveable` tersebut dan melakukan refactoring dengan menekan kombinasi tombol `Alt-Enter` di jendela *editor* Android Studio).

Compile dan jalankan kembali program, dengan memastikan posisi perangkat pada posisi **portrait**, kemudian klik beberapa kali tombol "Double" sedemikian rupa sehingga angka yang ditampilkan tidak lagi angka 1 yang merupakan nilai awal yang diset pada variabel `number` saat program pertama kali dijalankan.

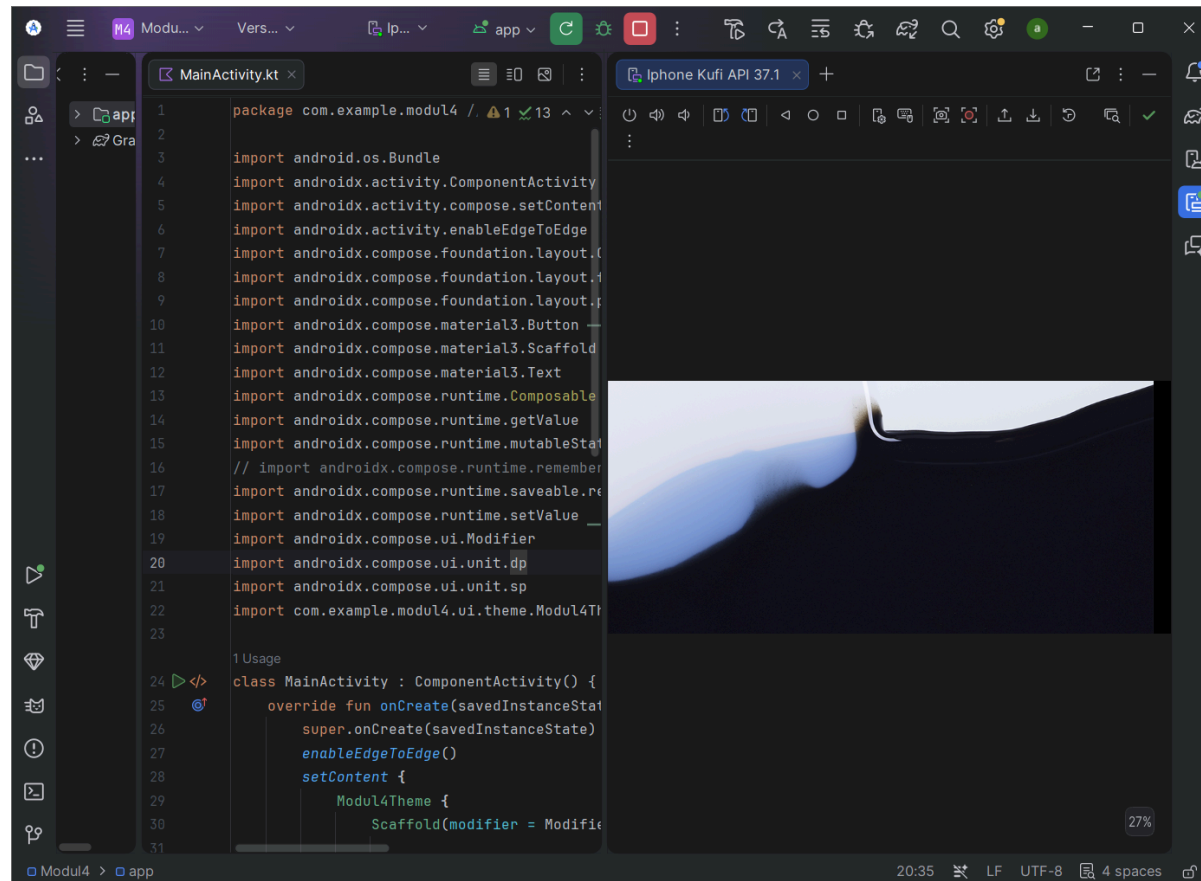
Catat angka yang ditampilkan setelah tombol "Double" diklik beberapa kali, ambil screenshot tampilan antarmukanya.

Jawaban:



3. Lakukan kembali perubahan konfigurasi dengan memutar posisi perangkat dari posisi *portrait* menjadi *landscape*, sehingga tampilan aplikasi akan disesuaikan oleh Android.
 - a. Catat angka yang ditampilkan kemudian ambil screenshot tampilan antar muka aplikasi di posisi landscape.

Jawaban: 16, mohon maaf tidak ada screenshot karena emulator nya error (saya memakai iphone, jadi emulator nya emang kadang suka tiba tiba error) setelah saya rotate ke landscape, tampilan yang keluar:



- b. Berapa angka yang ditampilkan? Mengapa terjadi demikian?

Jawaban: Angka yang ditampilkan tetap bertahan pada angka terakhir (16). Hal ini terjadi karena `rememberSaveable` secara otomatis menyimpan nilai *state* ke dalam sebuah objek *Bundle*, sehingga datanya tetap aman dan bisa dipulihkan meskipun sistem Android menghancurkan dan membuat ulang *Activity* saat terjadi rotasi layar.

- c. Apa fungsi dari konstruk `rememberSaveable` pada kode program Android yang membedakannya dari konstruk `remember`?

Jawaban: `rememberSaveable` berfungsi untuk mempertahankan data melewati proses perubahan konfigurasi (seperti rotasi layar) dan saat aplikasi dimatikan sementara oleh sistem untuk menghemat memori. Sementara itu, `remember` hanya mampu mempertahankan data selama proses *recomposition* (penggambaran ulang UI) biasa saat *Activity* tidak dihancurkan.

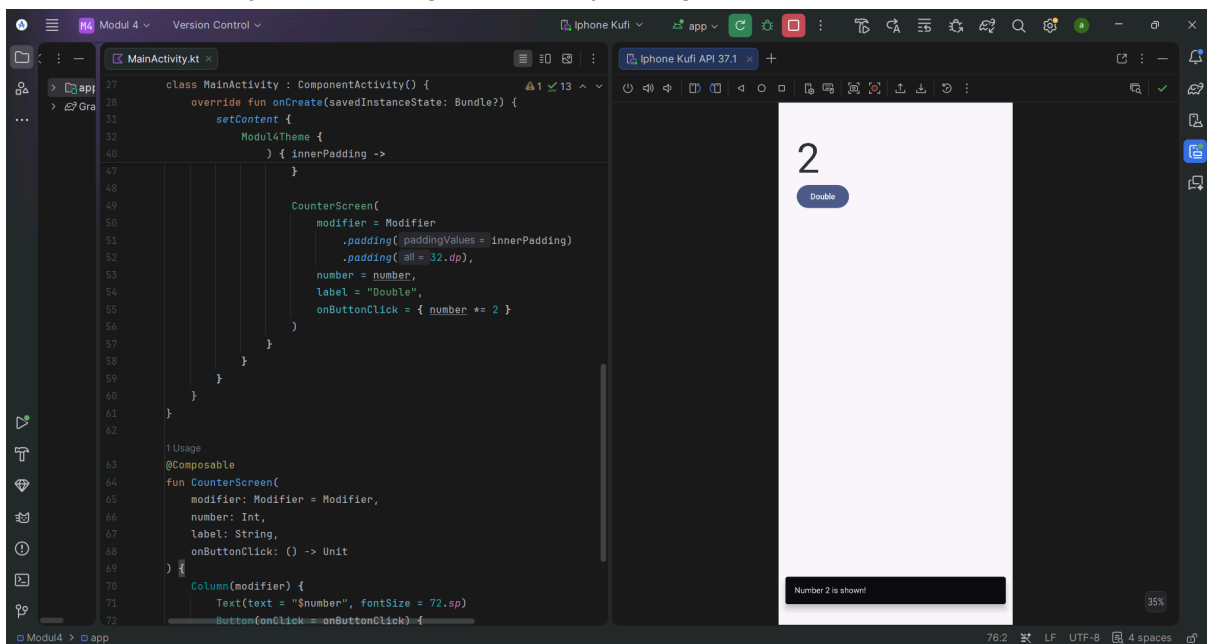
4. Beri contoh kasus nyata lain di mana penggunaan konstruk `rememberSaveable` ini tepat digunakan dalam aplikasi Android serta sebutkan manfaat yang diperoleh dari penerapannya!

Jawaban: Penggunaan `rememberSaveable` sangat tepat untuk menyimpan input teks pengguna pada *TextField* (seperti saat mengisi form registrasi login) atau untuk mempertahankan posisi *scroll* halaman yang panjang. Manfaat utamanya adalah meningkatkan pengalaman pengguna (UX), karena pengguna tidak akan kehilangan data isian mereka jika mereka tidak sengaja memutar ponsel (rotasi layar) atau saat aplikasi bergeser ke latar belakang.

Launched Effect

Percobaan 1

1. Jalankan program,
 - a. Pada saat pertama kali program dijalankan, tanpa mengklik *button* "Double" apa yang ditampilkan di dalam layar?
Jawaban: Menampilkan UI dengan angka 1 dan sebuah pesan *SnackBar* bertuliskan "Number 1 is shown!" di bagian bawah layar.
 - b. Mengapa pesan tersebut ditampilkan saat pertama kali aplikasi dijalankan tanpa ada interaksi apa pun dari pengguna?
Jawaban: Karena *LaunchedEffect* secara otomatis menjalankan blok kodenya saat komponen pertama kali masuk ke komposisi (saat layar dimuat).
 - c. Apa yang memicu pesan pada *SnackBar* tersebut ditampilkan? Pada bagian kode program yang mana?
Jawaban: Dipicu oleh eksekusi otomatis efek samping saat aplikasi dirender pertama kali, tepatnya pada baris `LaunchedEffect(number) { snackbarHostState.showSnackBar(...) }`
2. Klik *button* "Double" satu kali,
 - a. Apa yang terjadi?
Jawaban: Angka di layar berubah menjadi 2, dan *SnackBar* baru akan muncul bertuliskan "Number 2 is shown!".
 - b. Mengapa hal tersebut terjadi? Jelaskan!
Jawaban: Hal ini terjadi karena parameter kunci (*key*) dari *LaunchedEffect*, yaitu variabel *number*, mengalami perubahan nilai. Jika *key* berubah, *LaunchedEffect* akan membatalkan *coroutine* yang lama dan menjalankan ulang blok kodenya dengan nilai baru.



Percobaan 2

1. Jalankan program,

- a. Pada saat pertama kali program dijalankan, tanpa mengklik *button* "Double" apa yang ditampilkan di dalam layar?

Jawaban: Layar akan menampilkan angka 1 dan pesan *SnackBar* "Number 1 is shown!"

- b. Tunggu selama dua detik, kemudian periksa kembali apa yang ditampilkan pada layar! Apa yang tampil?

Jawaban: Angka pada layar berubah menjadi angka acak, dan muncul *SnackBar* baru yang memberitahukan bahwa angka acak tersebut ditampilkan.

- c. Apa yang memicu perubahan angka dan pesan yang ditampilkan pada *SnackBar* tersebut ditampilkan? Pada bagian kode program yang mana?

Jawaban: Pada bagian kode program yang mana? Dipicu oleh selesainya fungsi penundaan *coroutine* yang mengambil data secara asinkron. Berada pada bagian kode `LaunchedEffect(Unit) { number = Repo.getData() }`

2. Dari kedua percobaan terkait dengan side-effect dengan *LaunchedEffect*,

- a. Jelaskan apa perbedaan antara *LaunchedEffect(state)* dengan *LaunchedEffect(Unit)*?

Jawaban: *LaunchedEffect(state)* akan dijalankan kembali setiap kali variabel parameter kuncinya (seperti *number*) mengalami perubahan nilai. Sedangkan *LaunchedEffect(Unit)* hanya akan dijalankan tepat satu kali karena parameter kuncinya adalah konstanta yang tidak akan pernah berubah nilainya.

- b. Kapan operasi di dalam *LaunchedEffect(state)* dengan *LaunchedEffect(Unit)* dijalankan?

Jawaban: Keduanya sama-sama otomatis dijalankan pertama kali pada saat fungsi *composable* masuk ke dalam komposisi (tampil di layar).